

SparseViT: Revisiting Activation Sparsity for Efficient High-Resolution Vision Transformer

The IEEE/CVF Conference on Computer Vision and Pattern Recognition(CVPR) 2023

펀디멘탈팀

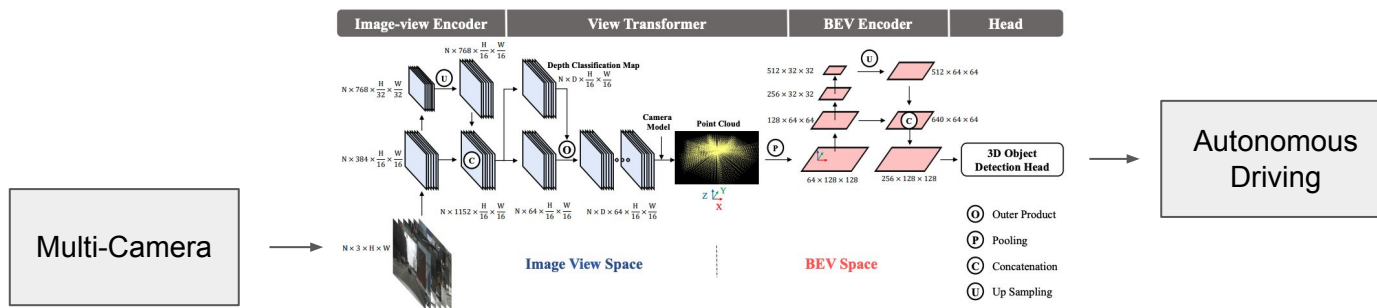
발표자: 신혜주

기여자: 임정환, 정영석

1. Introduction

Previous Work:

- : With the advancement of image sensors, high-resolution images become accessible
 - Comes at the cost of linearly-growing computational complexity(less deployable for resource-constrained applications)
- : BEVDet: High-Performance Multi-Camera 3D Object Detection in Bird-Eye-View [CVPR, 2021]
 - Modularly designed architecture with an image-view encoder, a view transformer(image view into BEV), a BEV encoder, and a task specific head for performing 3D object detection in the BEV space
 - The detection accuracy is degraded by more than 5% in mAP by reducing the height and width by 1.6X



ID	Input Resolution	BEV Resolution	mAP	NDS	mATE↓	mASE↓	mAOE↓	mAVE↓	mAAE↓	GFLOPs	FPS
A	704 × 256	0.8 Meter	0.312	0.392	0.691	0.272	0.523	0.909	0.247	215.3	15.6
B	1056 × 384	0.8 Meter	0.333	0.410	0.661	0.265	0.509	0.886	0.243	370.5	8.9
C	704 × 256	0.4 Meter	0.315	0.410	0.653	0.274	0.492	0.851	0.254	438.4	10.0
D	1056 × 384	0.4 Meter	0.348	0.417	0.644	0.266	0.475	0.916	0.264	593.6	7.1
E	1408 × 512	0.4 Meter	0.360	0.438	0.638	0.266	0.427	0.878	0.213	824.6	5.0

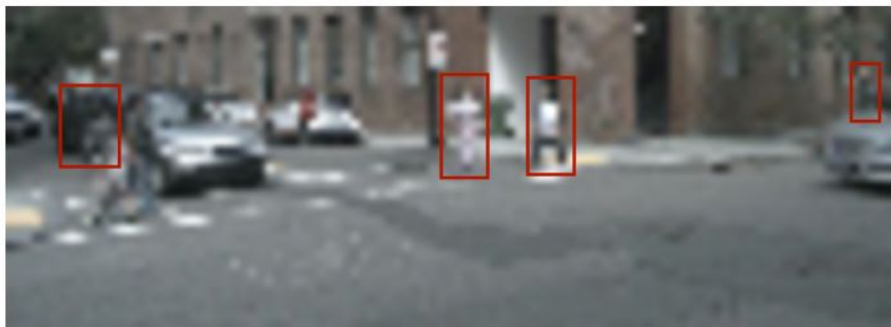
1. Introduction

Motivation:

- : Dropping details uniformly at all positions is clearly suboptimal as not all pixels are equally informative.
 - Within an image, the pixels that contain detailed object features are more important than the background pixel.
- Skipping computations for less-important regions (i.e., activation pruning)

Contribution:

- : *Revisits* the activation sparsity in the context of *window-based ViT*.
 - Re-implement the other layers in the swin transformer (i.e., FFNs and LNs) to also execute at the window level.
- : Achieve around *50% latency reduction* with *60% window activation sparsity*.
- : Make use of the *evolutionary search* to explore *the best per-layer pruning ratios* under a resource constraint
 - Different layers have different impacts on efficiency and accuracy, advocating for a non-uniform layerwise sparsity configuration.



(a) **Direct Downsample**: Lower Resolution (**0.5x**), Dense (**100%**)



(b) **Window Activation Pruning**: Higher Resolution (**1.0x**), Sparse (**25%**)

2. Related Work: ViT, Swin Transformer

Vision Transformer:

: An image is worth 16x16 words: Transformers for image recognition at scale [ICLR, 2021]

- Inspired by the Transformer scaling successes in NLP, applying a standard Transformer directly to images, with the fewest possible modification.
- Split an image into patches and provide the sequence of linear embeddings of these patches as an input.
- Intractable for Transformer on high-resolution images, as the computational complexity of its self-attention is quadratic to image size.

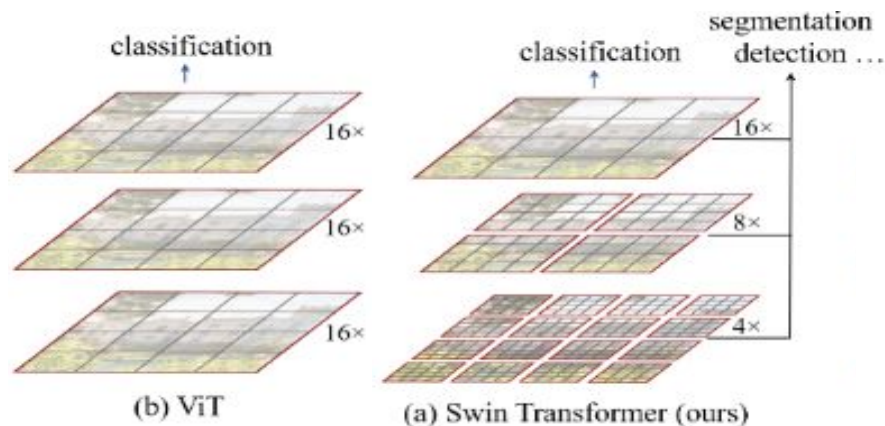
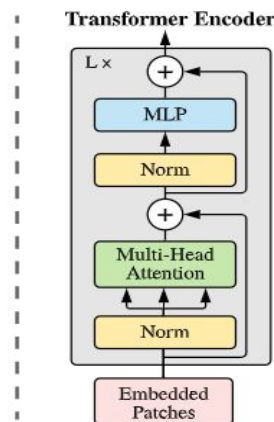
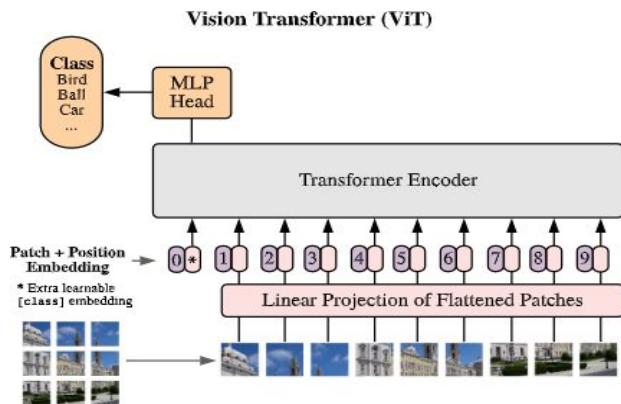
Window-based Transformer:

: Swin Transformer: Hierarchical Vision Transformer using Shifted Window [ICCV, 2021]

- Constructs *hierarchical feature maps* and has *linear computational complexity* to image size.
- Achieved by computing self-attention locally within non-overlapping windows that partition an image (outlined in red).
- The number of patches in each window is fixed, and thus the complexity becomes linear to image size.

$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C,$$

$$\Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC,$$



2. Related Work: ViT, Swin Transformer

Vision Transformer:

: An image is worth 16x16 words: Transformers for image recognition at scale [ICLR, 2021]

- Inspired by the Transformer scaling successes in NLP, applying a standard Transformer directly to images, with the fewest possible modification.
- Split an image into patches and provide the sequence of linear embeddings of these patches as an input.
- Intractable for Transformer on high-resolution images, as the computational complexity of its self-attention is quadratic to image size.

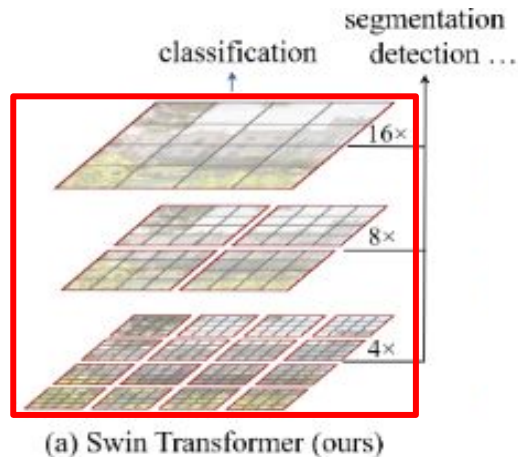
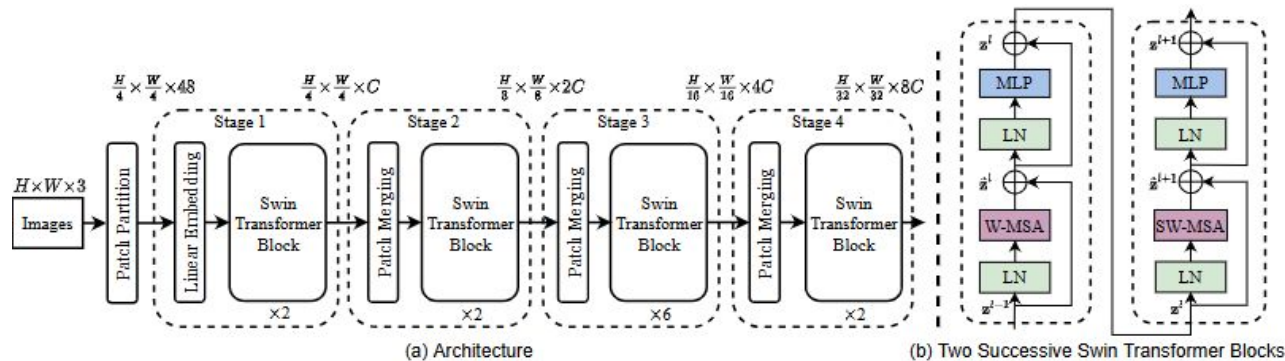
Window-based Transformer:

: Swin Transformer: Hierarchical Vision Transformer using Shifted Window [ICCV, 2021]

- Constructs *hierarchical feature maps* and has *linear computational complexity* to image size.
- Achieved by computing self-attention locally within non-overlapping windows that partition an image (outlined in red).
- The number of patches in each window is fixed, and thus the complexity becomes linear to image size.

$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C,$$

$$\Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC,$$



2. Related Work: ViT, Swin Transformer

Vision Transformer:

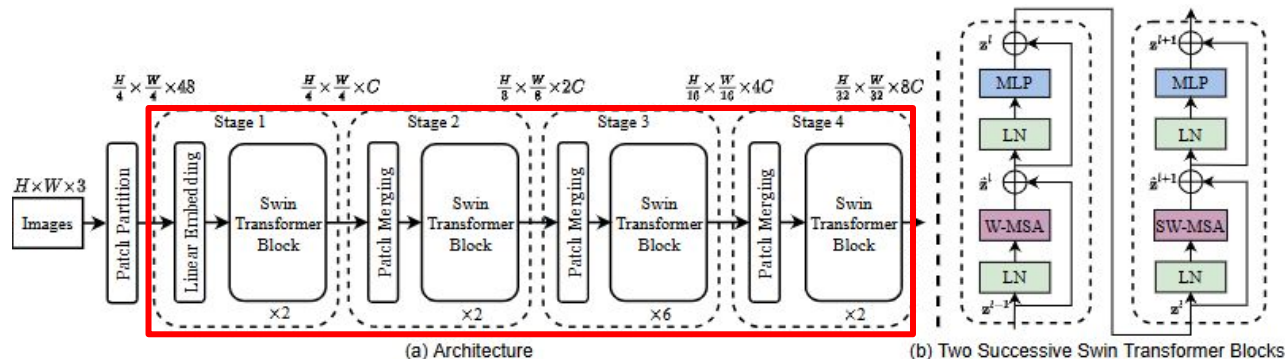
: An image is worth 16x16 words: Transformers for image recognition at scale [ICLR, 2021]

- Inspired by the Transformer scaling successes in NLP, applying a standard Transformer directly to images, with the fewest possible modification.
- Split an image into patches and provide the sequence of linear embeddings of these patches as an input.
- Intractable for Transformer on high-resolution images, as the computational complexity of its self-attention is quadratic to image size.

Window-based Transformer:

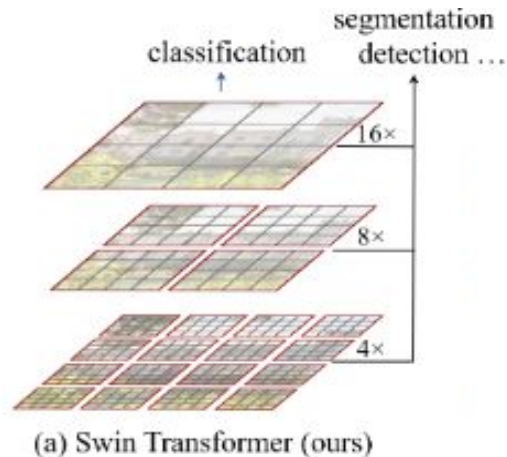
: Swin Transformer: Hierarchical Vision Transformer using Shifted Window [ICCV, 2021]

- Constructs *hierarchical feature maps* and has *linear computational complexity* to image size.
- Achieved by computing self-attention locally within non-overlapping windows that partition an image (outlined in red).
- The number of patches in each window is fixed, and thus the complexity becomes linear to image size.



$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C,$$

$$\Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC,$$



2. Related Work: ViT, Swin Transformer

Vision Transformer:

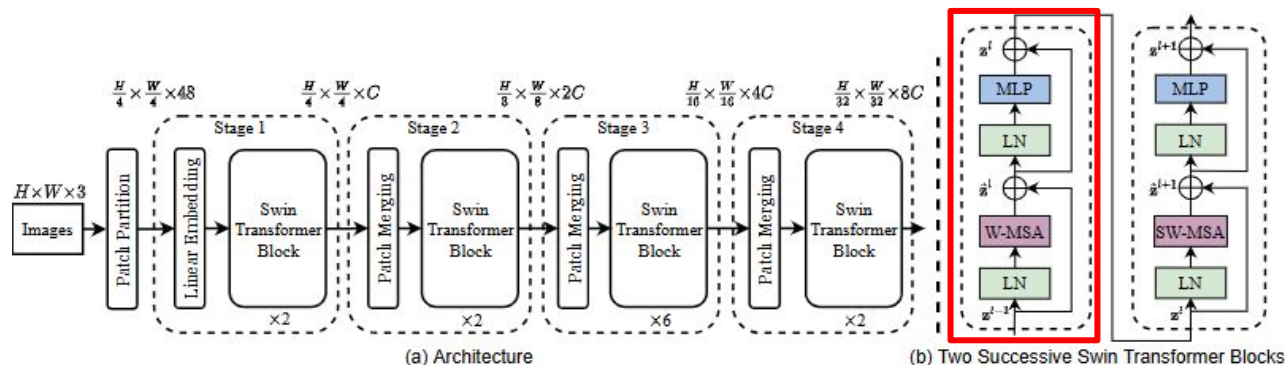
: An image is worth 16x16 words: Transformers for image recognition at scale [ICLR, 2021]

- Inspired by the Transformer scaling successes in NLP, applying a standard Transformer directly to images, with the fewest possible modification.
- Split an image into patches and provide the sequence of linear embeddings of these patches as an input.
- Intractable for Transformer on high-resolution images, as the computational complexity of its self-attention is quadratic to image size.

Window-based Transformer:

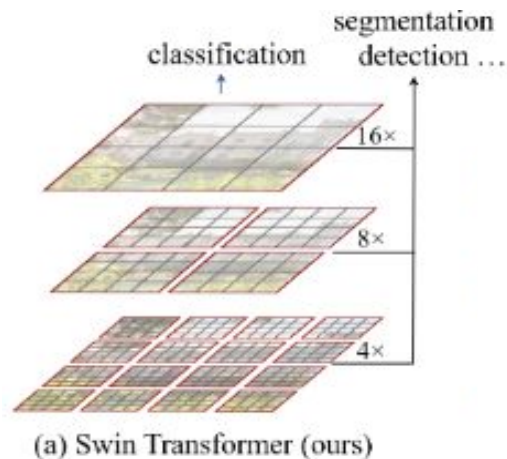
: Swin Transformer: Hierarchical Vision Transformer using Shifted Window [ICCV, 2021]

- Constructs *hierarchical feature maps* and has *linear computational complexity* to image size.
- Achieved by computing self-attention locally within non-overlapping windows that partition an image (outlined in red).
- The number of patches in each window is fixed, and thus the complexity becomes linear to image size.



$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C,$$

$$\Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC,$$



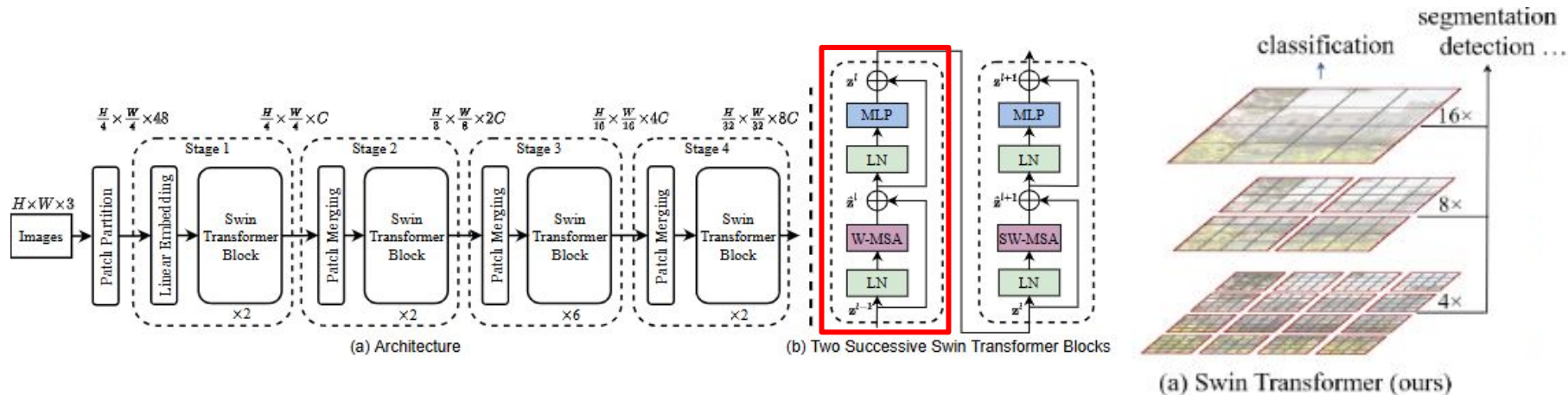
3. What is SparseViT ?

3.1. Swin Transformer Revisited

- : The original Swin Transformer applies *MHSA at the window level*, while *MLP and LN are applied to the entire feature map*.
 - Requires additional structuring before and after each MHSA, making window pruning more complicated.

Window-Wise Execution:

- : SparseViT modify the execution of MLP and LN to be window-wise as well.
 - All operations will be applied at the window level, making the mapping of the sparsity mask very easy.
 - This modification incurs a minimal accuracy drop of less than 0.1% due to padding, even without re-training.



3. What is SparseViT ?

3.2. Window Activation Pruning

: SparseViT define the importance of each window as its *L2 activation magnitude*.

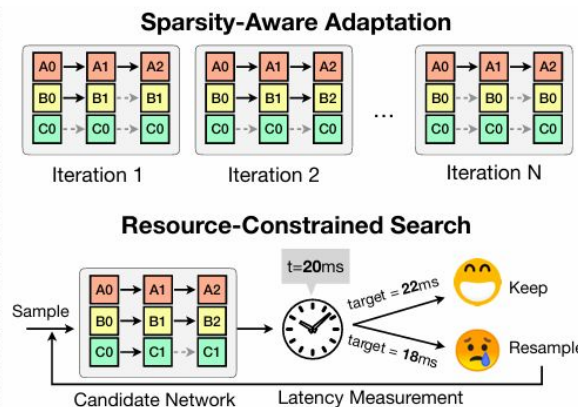
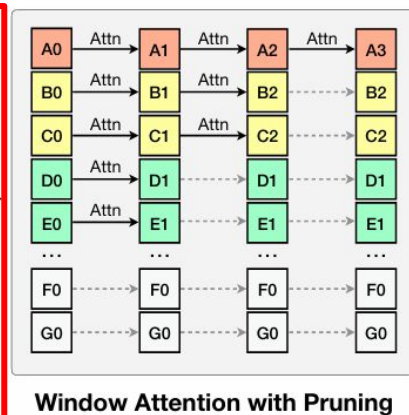
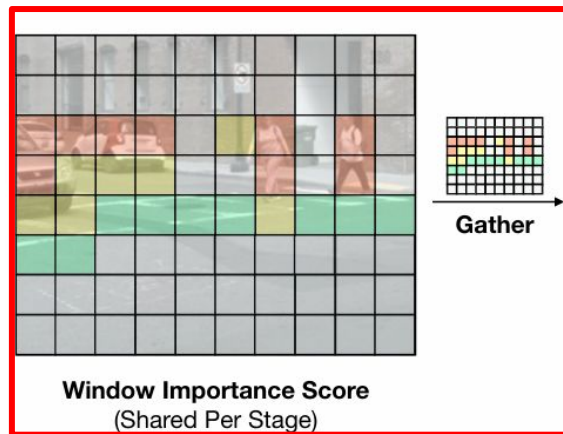
- Gather those windows with the highest importance score, once an activation sparsity ratio is given.
- Apply MHSA, FFN and LN only on these selected windows.
- Scatter outputs back to the workflow of window activation pruning.
- Duplicate the features of the unselected windows to mitigate information loss due to coarse-grained window pruning.

Shared Scoring:

: Unlike conventional weight pruning, importance scores are input-dependent and need to be computed during inference

- Can introduce significant overhead to the latency.

→ SparseViT *compute the window importance score only once per stage* and reuse it across all the blocks within the stage.



3. What is SparseViT ?

3.3. Mixed-Sparsity Configuration Search

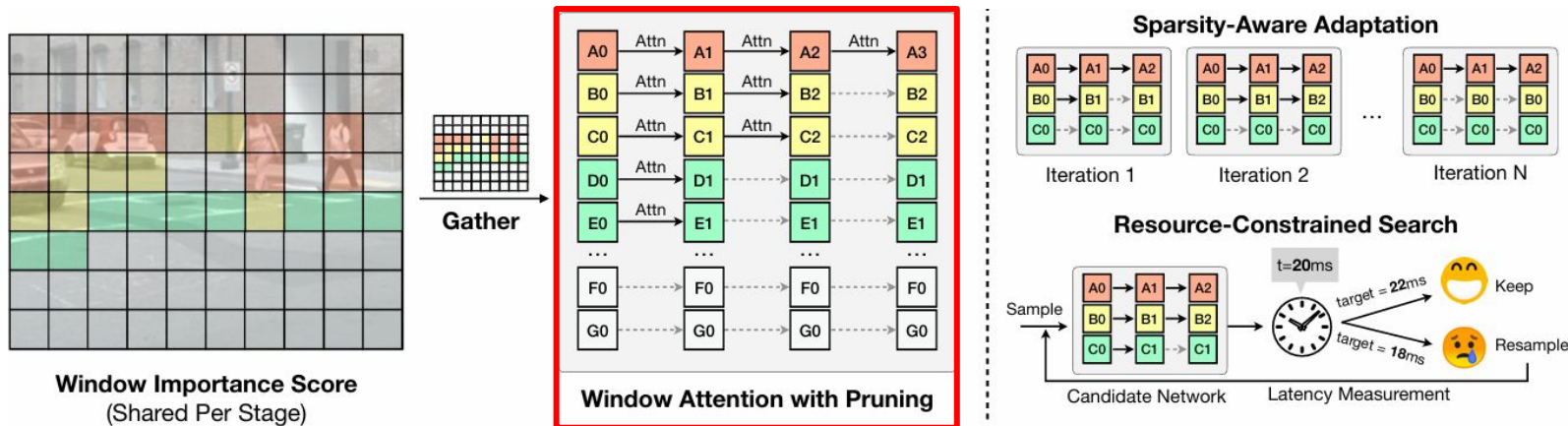
: A workflow that efficiently searches for the optimal mixed-sparsity pruning.

- Using a uniform sparsity level throughout a model may not be the best strategy because different layers have varying impacts on both accuracy and efficiency.
- It is more beneficial to apply more pruning to layers with lower sensitivity and higher costs.
 - However, manually exploring layerwise sparsity can be a time-consuming and error-prone task.

Search Space:

: The search space for mixed-sparsity activation pruning.

- For each Swin block, SparseViT allow the sparsity ratio to be chosen from $\{0\%, 10\%, \dots, 80\%\}$.



3. What is SparseViT ?

3.3. Mixed-Sparsity Configuration Search

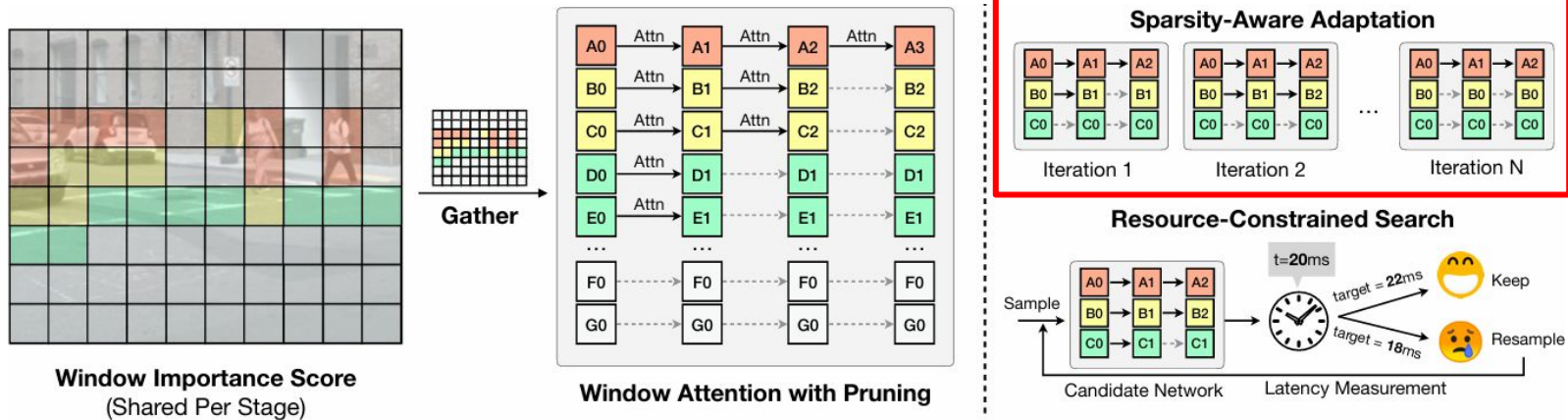
: A workflow that efficiently searches for the optimal mixed-sparsity pruning.

- Using a uniform sparsity level throughout a model may not be the best strategy because different layers have varying impacts on both accuracy and efficiency.
- It is more beneficial to apply more pruning to layers with lower sensitivity and higher costs.
 - However, manually exploring layerwise sparsity can be a time-consuming and error-prone task.

Sparsity-Aware Adaptation (SAA):

: Adapting the original model, which was trained with only dense activations, by *randomly sampling layerwise activation sparsity* and *updating the model accordingly at each iteration*.

→ SparseViT can obtain a more accurate estimate of the performance of different sparsity configurations without full retraining.



3. What is SparseViT ?

3.3. Mixed-Sparsity Configuration Search

: A workflow that efficiently searches for the optimal mixed-sparsity pruning.

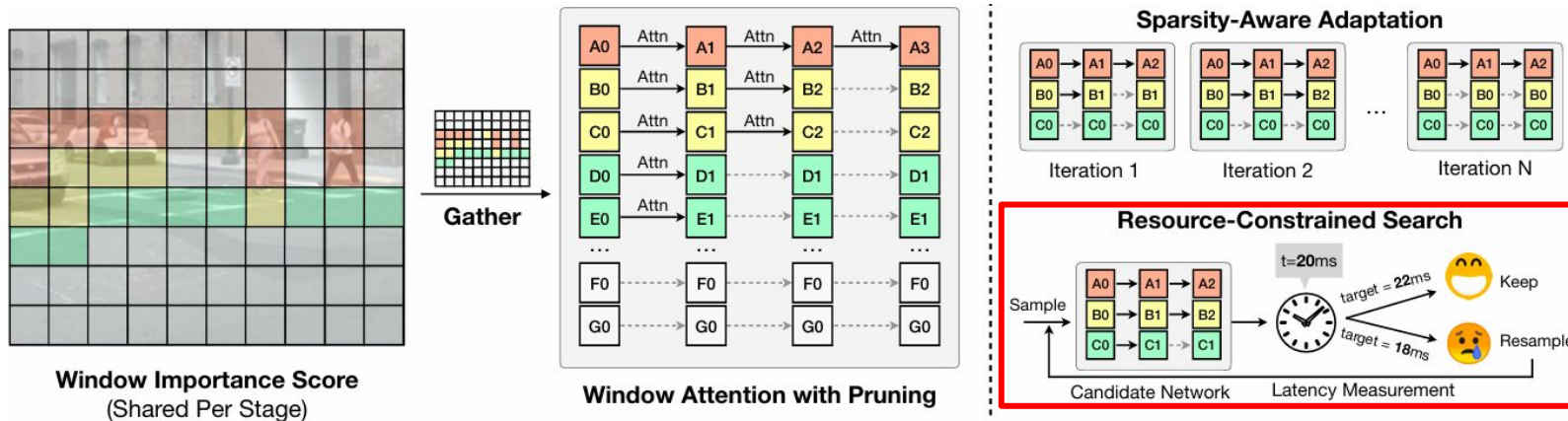
- Using a uniform sparsity level throughout a model may not be the best strategy because different layers have varying impacts on both accuracy and efficiency.
- It is more beneficial to apply more pruning to layers with lower sensitivity and higher costs.
- However, manually exploring layerwise sparsity can be a time-consuming and error-prone task.

Resource-Constrained Search:

: With SAA, SparseViT can proceed to search for *the best sparsity configurations within specified resource constraints*.

- Two types of resource constraints: hardware-agnostic computational cost (i.e., #MACs), and hardware-dependent latency.

: To perform the sparsity search, SparseViT adopt the *evolutionary algorithm*.



4. Experiment Results

Latency Measurement:

- : The latency measurements are obtained using a single NVIDIA RTX A6000 GPU.
- Perform 500 inference steps as a warm-up and subsequently measure the latency for another 500 steps.
- Report the average latency of the middle 250 measurements out of the 500 to minimize the variance.

4.1.1. Monocular 3D Object Detection

- : The nuScenes benchmark dataset, which includes 1,000 scenes, was used in this experiment. Only camera inputs were employed.
- : The input images to 256×704 and train the model for 20 epochs.
- : Table 1 show that SparseViT achieves the same accuracy as Swin-T with 1.8× lower #MACs and 1.5× faster inference latency.

Backbone	Resolution	Width	#MACs (G)	Latency (ms)	mAP _↑	mATE _↓	mASE _↓	mAOE _↓	mAVE _↓	mAAE _↓	NDS _↑
Swin-T	256×704	1×	140.8	36.4	31.2	69.1	27.2	52.3	90.9	24.7	39.2
SparseViT (Ours)	288×792	1×	113.9	34.5	32.0	72.8	27.2	53.8	79.4	25.7	40.1
Swin-T (R224)	224×616	1×	78.5	23.0	29.9	71.8	27.4	60.9	79.0	26.0	38.4
Swin-T (W0.6×	256×704	0.6×	56.0	22.6	29.9	69.9	27.5	59.9	81.4	25.8	38.5
SparseViT (Ours)	256×704	1×	78.4	23.8	31.2	70.9	27.5	58.7	83.1	27.2	38.9
Swin-T (R192)	192×528	1×	67.1	18.7	28.7	74.3	27.9	59.5	76.7	27.8	37.7
Swin-T (W0.4×	256×704	0.4×	20.4	17.6	27.6	74.2	27.9	63.4	91.0	26.2	35.5
SparseViT (Ours)	256×704	1×	58.6	18.7	30.0	72.0	27.5	59.7	81.7	26.6	38.3

Table 1. Results of monocular 3D object detection on nuScenes.

4. Experiment Results

Latency Measurement:

- : The latency measurements are obtained using a single NVIDIA RTX A6000 GPU.
- Perform 500 inference steps as a warm-up and subsequently measure the latency for another 500 steps.
- Report the average latency of the middle 250 measurements out of the 500 to minimize the variance.

4.1.2. 2D Instance Segmentation

- : The COCO dataset, which contains 118k/5k training/validation images, was used for 2D instance segmentation.
- : Use the default input resolution of 640×640 and train the model for 36 epochs.
- : As in Table 2, SparseViT consistently outperforms the baseline with less computation across various input resolutions from 512×512 to 640×640.

Backbone	Resolution	Width	#MACs (G)	Latency (ms)	AP^{bbox}	AP_{50}^{bbox}	AP_{75}^{bbox}	AP^{mask}	AP_{50}^{mask}	AP_{75}^{mask}
Swin-T	640×640	1×	161.8	46.6	42.0	63.3	45.7	38.3	60.3	40.9
Swin-T (R576)	576×576	1×	149.5	41.3	41.0	62.1	44.9	37.2	59.0	39.6
Swin-T (W0.9×	640×640	0.9×	122.3	41.8	40.4	61.9	43.8	37.1	58.9	39.8
SparseViT (Ours)	672×672	1×	139.5	41.3	42.4	63.3	46.4	38.5	60.3	41.3
Swin-T (R544)	544×544	1×	119.8	34.8	40.5	61.2	43.8	36.8	58.2	39.1
Swin-T (W0.8×	640×640	0.8×	90.5	35.9	39.4	60.7	42.8	36.4	57.9	38.8
SparseViT (Ours)	672×672	1×	116.5	34.1	41.6	62.5	45.5	37.7	59.4	40.2
Swin-T (R512)	512×512	1×	117.5	32.9	39.6	60.1	43.4	36.0	57.0	38.2
Swin-T (W0.6×	640×640	0.6×	63.4	31.7	38.7	60.2	41.6	35.7	57.0	38.0
SparseViT (Ours)	672×672	1×	105.9	32.9	41.3	62.2	44.9	37.4	59.1	39.7

Table 2. Results of 2D instance segmentation on COCO.

4. Experiment Results

Latency Measurement:

- : The latency measurements are obtained using a single NVIDIA RTX A6000 GPU.
- Perform 500 inference steps as a warm-up and subsequently measure the latency for another 500 steps.
- Report the average latency of the middle 250 measurements out of the 500 to minimize the variance.

4.1.3. 2D Semantic Segmentation

- : Benchmark dataset for 2D semantic segmentation is Cityscape, which consists of over 5k high-quality, fully annotated images with pixel-level semantic labels for 30 object classes, including road, building, car, and pedestrian.
- : Use the default input resolution of 1024×2048 and train the model for 90k epochs.
- : Based on Table 3, SparseViT model attains comparable segmentation accuracy to the baseline while delivering a speedup of 1.3 $\times$.

Backbone	Resolution	Latency (ms)	mIoU
Swin-L	1024×2048	329.5	83.3
Swin-L (R896)	896×1792	256.5	82.8
SparseViT (Ours)	1024×2048	250.6	83.2

Table 3. Results of 2D semantic segmentation on Cityscapes.

5. Analysis

Window vs. Token Pruning

: Window pruning is more effective than token pruning.

- SparseViT uses window-level pruning.
- Compared to token pruning (DynamicViT[39]):
 - . 0 accuracy drop vs. 0.4 mAP drop
 - . Lower latency and better memory locality

	#MACs (G)	Latency (ms)	mAP Drop
DynamicViT	98.6	33.3	-0.1
DynamicViT	91.7	32.6	-0.4
SparseViT	78.4	23.8	0.0

Table 4. Window pruning (SparseViT) is more efficient and effective than learnable token pruning (DynamicViT).

High-Resolution Input Advantage

: Pruning from higher resolution matters (Start High, Prune More).

- High-resolution input
 - better detail retention More uninformative windows to prune
- Best performance at 256×704 input (Table 5)

Input Resolution	#MACs (G)	Latency (ms)	mAP
192×528	67.1	18.7	28.7
224×616	64.9	19.1	29.7
256×704	58.6	18.7	30.0

Table 5. Starting from a high-resolution input and pruning more is better than starting from a low-resolution input and pruning less.

5. Analysis

Scoring Strategy

- : L2 magnitude-based scoring is simple and effective.
- L2 Magnitude > Learnable Score
 - . Simpler L2-based scoring performs better
 - . Less latency and fewer MACs (Table 8)

Scoring	#MACs (G)	Latency (ms)	mAP Drop
Learnable	89.6	33.1	-0.6
L2 Magnitude	78.4	23.8	0.0

Table 8. L2 magnitude-based scoring is simple and effective, achieving a better accuracy-efficiency trade-off than learnable scoring.

Shared Scoring

- : Shared scoring is better than independent scoring.
- Stage-wise Shared Scoring
- Sharing scores per stage is efficient
- Maintains performance and improves latency (Table 9)

Scoring	Sparsity	#MACs (G)	Latency (ms)	mAP
Independent	0.5	70.68	23.3	30.08
Shared	0.5	70.68	23.0	30.17

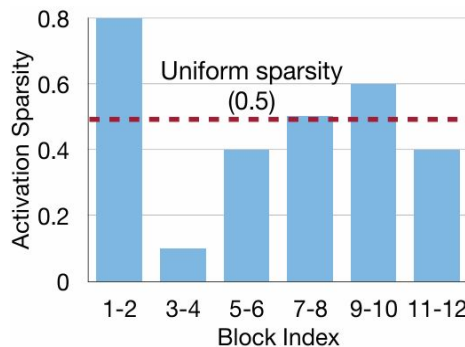
Table 9. Shared scoring per stage reduces the cost of score calculation, leaving room for effective computation and offering a better accuracy-latency trade-off than independent scoring per block.

5. Analysis

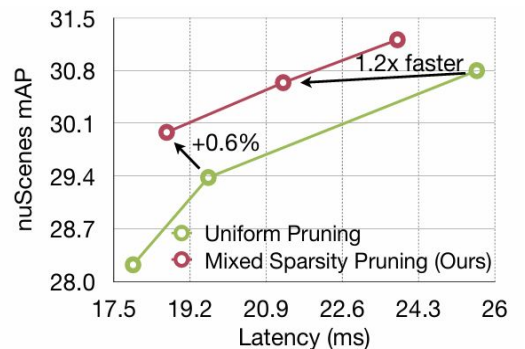
Mixed vs. Uniform Sparsity

: Mixed-sparsity pruning is better than uniform pruning.

- Mixed Sparsity Strategy
 - . Apply non-uniform pruning:
 - Aggressive early pruning
 - Conservative later pruning
 - . 0.6% better accuracy and 1.2× faster (b)



(a) Blockwise Sparsity Distribution

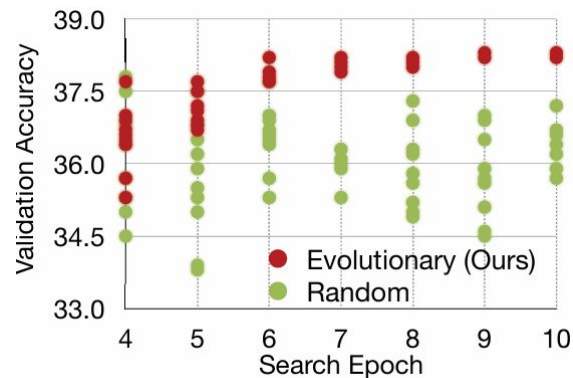


(b) Mixed Sparsity Pruning vs. Uniform Pruning

Evolutionary Search

: Evolutionary search is better than random search.

- Smarter Search with Evolution
- Evolutionary search outperforms random search
- Converges faster with better accuracy (c)



(c) Evolutionary Search vs. Random Search

5. Analysis

Sparsity-Aware Adaptation (SAA)

- : Sparsity-aware adaptation offers a better accuracy proxy.
 - Better Proxy for Accuracy
 - SAA gives better prediction of post-fine tuning accuracy
 - Conventional methods underestimate certain pruning strategies (Table 6)

SAA	Sparsity Ratios	mAP w/o FT	mAP w/ FT
	(0.5, 0, 0, 0)	28.31	31.40
	(0, 0.5, 0, 0)	30.93	31.37
✓	(0.5, 0, 0, 0)	30.83	31.45
	(0, 0.5, 0, 0)	30.63	31.44

Table 6. Sparsity-aware adaptation (SAA) improves the correlation between the accuracy before and after finetuning (FT).

Convergence with SAA

- : Sparsity-aware adaptation improves the accuracy.
 - SAA Improves Training
 - Best results when fine tuning with SAA
 - Likely acts as implicit knowledge distillation (Table 7)

	mAP
(a) Training from scratch	29.67
(b) Finetuning from a pre-trained Swin-T	30.83
(c) Finetuning from a pre-trained Swin-T with SAA	31.21

Table 7. Sparsity-aware adaptation improves the convergence.

5. Analysis

Qualitative Results

: SparseViT learns to keep important foreground windows.

- SparseViT Learns Foreground
- Retains windows around important objects
- Object contours are preserved (Figure 5)



Figure 5. SparseViT effectively prunes irrelevant background windows while retaining informative foreground windows. Each window's color corresponds to the number of layers it is executed. Brighter colors indicate that the model has executed the window in more layers.

6. Conclusion

Motivation

- Activation pruning is powerful for preserving high-resolution information.
- Recent **window-based Vision Transformers (ViTs)** offer new opportunities to leverage activation sparsity efficiently.

Contribution

- We propose **SparseViT**, a novel approach that:
 - Introduces **Sparsity-Aware Adaptation (SAA)** for robust training under sparse activations.
 - Achieve around **50% latency reduction** with 60% window activation sparsity.
 - Uses **evolutionary search** to discover optimal layer-wise sparsity configurations.

Results

- Achieved **real speedups** with **minimal or no accuracy loss**:
 - **1.5×** in monocular 3D object detection
 - **1.4×** in 2D instance segmentation
 - **1.3×** in 2D semantic segmentation

Impact

- Demonstrates that **activation pruning can be both efficient and accurate** in ViTs.
- We hope this inspires future research toward high-resolution, computation-efficient vision models.

Thank you for listening!